

Problem 1: Word Embeddings from IIT Jodhpur Data

P1 ▶ Corpus File Size

0.168 MB (175,925 bytes) Documents: 36 | Tokens: 25,964 | Vocab: 3,547

P1 ▶ Corpus Curation & Preprocessing Steps

- Step 1: Data Collection - Scraped data from IIT Jodhpur website (Departments, Academics, Research, Faculty profiles, Announcements) using Requests + BeautifulSoup.
- Step 2: Boilerplate Removal - Removed headers, footers, and non-English lines using ASCII character proportion heuristic (>70% ASCII = English).
- Step 3: Text Cleaning - Removed HTML tags, LaTeX fragments, excessive punctuation, numbers and non-textual content using regular expressions.
- Step 4: Lowercasing - Converted all text to lowercase for vocabulary normalization.
- Step 5: Tokenization - Tokenized sentences using NLTK's sent_tokenize, then into words using word_tokenize.
- Step 6: Frequency Filtering - Filtered tokens with frequency < 2 (used as min_count in Word2Vec training) to reduce noise from rare/misspelled tokens.

P1 ▶ Top-10 Words (Frequency-wise)

1. the	1,552	2. of	1,088	3. and	773	4. to	640	5. for	542
6. in	482	7. a	468	8. be	295	9. student	281	10. by	232

P1 ▶ Interesting Analogy Found

semester : examination :: thesis : defense

Interpretation: Just as a semester culminates in an examination, a PhD thesis culminates in a 'defense' (viva-voce). The model also resolved:

UG : B.Tech :: PG : M.Tech and student : learning :: faculty : teaching

Trained using Word2Vec Skip-gram (vector_size=300, window=5, negative_sampling=5, min_count=2, epochs=20) on the IITJ corpus.

student —

-0.0332, 0.0768, -0.0540, 0.0166, 0.1287, -0.0581, 0.1325, 0.3132, 0.0152, -0.2531,
0.1639, -0.3321, 0.0136, 0.1670, -0.0625, 0.0303, 0.2647, 0.0302, 0.0870, -0.2013,
-0.0363, -0.0993, -0.0402, -0.0409, 0.2379, -0.0981, -0.0079, 0.1399, -0.0009, -0.1955,
0.2793, 0.2148, 0.2573, 0.0491, 0.0061, -0.0867, 0.0691, -0.1670, -0.0555, -0.2244,
0.0210, -0.0878, 0.1224, -0.1436, 0.2938, 0.0395, -0.0657, 0.1391, 0.2132, 0.1279,
-0.0335, 0.2173, -0.0258, 0.4945, 0.0994, 0.3428, 0.1370, 0.0780, 0.0195, 0.0295, 0.0804,
0.2847, 0.1784, 0.0062, -0.2059, 0.1322, 0.0449, 0.0781, -0.0034, 0.1986, -0.0058, 0.1291,
0.1675, -0.2651, 0.2191, 0.0783, -0.1050, 0.0214, -0.1892, -0.0813, -0.0829, 0.1267,
-0.0512, 0.3153, 0.1579, 0.1999, 0.0993, -0.1966, 0.1001, 0.3156, 0.1392, -0.0833,
-0.1254, 0.1173, 0.0844, -0.0081, 0.0483, -0.1338, 0.2399, -0.0644, -0.0824, -0.0188,
-0.1603, 0.0743, -0.0908, -0.1876, -0.0464, 0.1061, -0.0887, -0.0982, -0.2921, 0.0763,
0.0154, 0.2368, -0.0027, 0.0362, 0.0480, -0.0192, 0.0926, -0.3540, 0.1749, -0.1050,
0.0442, -0.0509, -0.0589, 0.0479, -0.1219, -0.2245, 0.2742, 0.2679, 0.0255, -0.1560,
-0.0653, -0.1290, 0.0717, 0.4318, -0.0419, -0.1544, -0.1650, -0.0917, 0.0400, -0.2401,
-0.1098, -0.0277, -0.1026, -0.2030, -0.0642, 0.0018, 0.0783, -0.2667, -0.0960, -0.1297,
0.0692, 0.0572, 0.0013, 0.0187, -0.2271, -0.0971, -0.0112, -0.0152, -0.1142, 0.0045,
-0.0459, 0.0722, 0.1139, 0.3181, 0.0746, -0.0624, -0.0267, 0.1068, -0.2320, -0.0301,
-0.0596, 0.1046, -0.0659, 0.0053, -0.0784, -0.0829, -0.0141, 0.1260, 0.0115, 0.0642,
0.0477, 0.0450, -0.1304, 0.1254, 0.1332, 0.0799, 0.1418, 0.0242, 0.3049, -0.0933, -0.1628,
0.1725, -0.0006, -0.3005, 0.0397, 0.0649, -0.1059, 0.0509, -0.0735, -0.0505, -0.1170,
0.0720, 0.1789, -0.0574, 0.1407, 0.1869, 0.1582, 0.0563, -0.1839, 0.0663, -0.0328,
-0.2670, -0.0342, -0.0568, 0.0133, -0.4136, -0.1195, -0.1311, -0.0663, 0.0298, -0.1353,
-0.0021, -0.1324, -0.1946, -0.0774, 0.1757, -0.0338, -0.0149, 0.1579, -0.0143, 0.0173,
0.1348, -0.1733, 0.0279, -0.1201, -0.0066, 0.0334, -0.1462, 0.1127, -0.1394, -0.1055,
0.1111, 0.0273, -0.0760, 0.2463, -0.1008, 0.2680, 0.3570, 0.0466, -0.1980, 0.1632,
-0.0232, -0.2050, -0.0057, 0.1749, -0.0948, -0.5011, -0.1659, -0.1496, -0.0508, 0.1911,
-0.1177, -0.4334, 0.1428, 0.0994, 0.0340, -0.0555, 0.0211, 0.0251, 0.0851, 0.2171, 0.1014,
-0.0047, 0.0674, 0.1998, 0.0234, 0.0488, -0.1421, 0.0388, -0.0111, -0.0579, 0.3688,
0.0446, -0.2802, -0.1295, -0.0330, -0.0471, 0.1167, 0.2951, 0.0938, 0.1564, -0.0060,
0.1240, 0.1138, 0.0743, 0.0030, 0.0674, -0.2208

Problem 2: Character-Level Name Generation using RNN Variants6

Dataset (Task-0)

1,000 unique Indian names curated from North Indian, South Indian, Bengali, Gujarati, Marathi, and pan-Indian name pools (stored in TrainingNames.txt).

Model Architectures (Task-1) — All implemented from scratch using PyTorch

Model	Architecture Description	Params	Size
Vanilla RNN	Manual VanillaRNNCell (tanh activation). 2 stacked layers. No gating mechanism. Simple recurrence: $h_t = \tanh(W_{ih} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$	61,393	0.24 MB
Bi-LSTM	Manual LSTMCell with 4 gates (i/f/o/g). Forward + Backward passes independently stacked (2 layers each). Outputs concatenated for projection.	442,193	1.69 MB
Attention +RNN	Manual LSTMCell (2 layers) + Luong general attention. At each step: $score = h_t^T \cdot W \cdot h_j$; $context = \sum \alpha_j \cdot h_j$; $output = \tanh(W_c \cdot [context; h_t])$	271,185	1.04 MB

Hyperparameters (shared across all models)

Embedding dim: 32 | Hidden size: 128 | Layers: 2 | Learning rate: 0.003 | Epochs: 100 (Vanilla) / 60 (Bi-LSTM, Attention) | Dropout: 0.2 (Vanilla), 0.4 (Bi-LSTM), 0.3 (Attention) | Optimizer: Adam | LR scheduler: StepLR ($\gamma=0.5$)

Quantitative Evaluation (Task-2)

Model	Novelty Rate	Diversity	Sample Names Generated
Vanilla RNN	24.6%	80.4%	Chaitanya, Namit, Vasuki, Sahan, Shubh, Devi
Bi-LSTM	100.0%	97.6%	Ojw, Yegi, Tookst, Yotas, Gokgs, Hgoy
Att+RNN	100.0%	100.0%	Diverse but less realistic character strings

P2 ▶ Which model works better, and why?

Vanilla RNN performs best for realistic name generation on this dataset.

- Vanilla RNN excels because Indian names follow short, consistent phonotactic patterns (avg 6.1 chars). Its simple recurrence $h_t = \tanh(W_{ih} \cdot x + W_{hh} \cdot h_{t-1})$ captures these local character transitions effectively without over-parameterization.
- Bi-LSTM (442K params) massively over-fitted the 1000-name training set during teacher forcing (training loss $\rightarrow 0.003$) making its autoregressive generation unreliable, since the bidirectional context used during training is unavailable at inference time.
- Attention+RNN had the highest diversity (100%) but generated less realistic names, as the attention mechanism requires longer sequences to learn meaningful alignments; with short names (avg ~ 6 chars) there is insufficient context to attend over.

Conclusion: For short, domain-specific sequences with a small training set, simpler models generalize better than over-parameterized ones.

P2 ▶ Vanilla RNN — Number of Parameters & Model Size

- Trainable Parameters: 61,393
Breakdown: embedding (1,568) + rnn_cell_0.W_ih (4,096) + rnn_cell_0.W_hh (16,384) + rnn_cell_0.b (128) + rnn_cell_1.W_ih (16,384) + rnn_cell_1.W_hh (16,384) + rnn_cell_1.b (128) + fc_out.weight (6,272) + fc_out.bias (49)
- Model Size on Disk: 0.24 MB (checkpoint including optimizer state)